

PPD - Laborator 1 - Documentatie

C++ - Dinamic

C++	Tip Matrice	Tip Filtru	Nr Threads	Tip Thread	Timp Executie
1)	N=M=10	n=m=3	-	Static	0.03
			4	Orizontal	0.44
				Vertical	0.74
2)	N=M=1000	n=m=5	-	Static	116.23
			2	Orizontal	53.06
				Vertical	58.75
			4	Orizontal	30.49
				Vertical	41.01
			6	Orizontal	27.66
				Vertical	27.82
			16	Orizontal	28.7
				Vertical	30.42
3)	N=10 M=10000	n=m=5	-	Static	17.86
			2	Orizontal	9.65
				Vertical	10.78
			4	Orizontal	6.88
				Vertical	7.02
			8	Orizontal	6.56
				Vertical	6.25
			16	Orizontal	5.14
				Vertical	5.24
4)	N=10000 M=10	n=m=5	-	Static	18.47
			2	Orizontal	9.72
				Vertical	10.22
			4	Orizontal	6.67
				Vertical	8.7
			8	Orizontal	6.19
				Vertical	6.91

C++	Tip Matrice	Tip Filtru	Nr Threads	Tip Thread	Timp Executie
			16	Orizontal	4.92
				Vertical	5.11
5)	N=10000 M=10000	n=m=5	-	Static	21595.57
			2	Orizontal	9403.3
				Vertical	16621.04
			4	Orizontal	7025.21
				Vertical	13388.08
			8	Orizontal	2851.25
				Vertical	6075.47
			16	Orizontal	1836.21
				Vertical	3301.31

C++ - Alocare Statica

C++	Tip Matrice	Tip Filtru	Nr Threads	Tip Thread	Timp Executie
1)	N=M=10	n=m=3	-	Static	0.01
			4	Orizontal	0.89
				Vertical	1.27
2)	N=M=1000	n=m=5	-	Static	13.39
			2	Orizontal	7.23
				Vertical	7.02
			4	Orizontal	5.75
				Vertical	5.37
			6	Orizontal	3.87
				Vertical	4.8
			16	Orizontal	3.98
				Vertical	4.01
3)	N=10 M=10000	n=m=5	-	Static	10.37
			2	Orizontal	6.3
				Vertical	6.38
			4	Orizontal	4.06
				Vertical	4.26
			8	Orizontal	3.87
				Vertical	3.83

C++	Tip Matrice	Tip Filtru	Nr Threads	Tip Thread	Timp Executie
			16	Orizontal	3.96
				Vertical	4.47
4)	N=10000 M=10	n=m=5	-	Static	11.74
			2	Orizontal	7.18
				Vertical	7.28
			4	Orizontal	4.98
				Vertical	5.08
			8	Orizontal	3.85
				Vertical	4.38
			16	Orizontal	3.53
				Vertical	3.88
5)	N=10000 M=10000	n=m=5	-	Static	57757.2
			2	Orizontal	24687.37
				Vertical	33042.67
			4	Orizontal	18806.9
				Vertical	26163.2
			8	Orizontal	21638.2
				Vertical	27516.6
			16	Orizontal	7493.24
				Vertical	8552.51

Java

Java	Tip Matrice	Tip Filtru	Nr Threads	Tip Thread	Timp Executie
1)	N=M=10	n=m=3	-	Static	0.91
			4	Orizontal	2.42
				Vertical	2.83
2)	N=M=1000	n=m=5	-	Static	52.51
			2	Orizontal	40.71
				Vertical	49.81
			4	Orizontal	33.9
				Vertical	43.38
			6	Orizontal	40.69
				Vertical	53.7

Java	Tip Matrice	Tip Filtru	Nr Threads	Tip Thread	Timp Executie
			16	Orizontal	38.25
				Vertical	56.85
3)	N=10 M=10000	n=m=5	-	Static	25.17
			2	Orizontal	11.34
				Vertical	15.48
			4	Orizontal	7.65
				Vertical	22.22
			8	Orizontal	10.17
				Vertical	24.7
			16	Orizontal	11.58
				Vertical	16.42
4)	N=10000 M=10	n=m=5	-	Static	16.1
			2	Orizontal	9.38
				Vertical	21.99
			4	Orizontal	7.74
				Vertical	32.23
			8	Orizontal	10.43
				Vertical	24.25
			16	Orizontal	8.87
				Vertical	32.63
5)	N=10000 M=10000	n=m=5	static	Static	5500.46
			2	Orizontal	2803.2
				Vertical	2847.07
			4	Orizontal	2357.96
				Vertical	2241.65
			8	Orizontal	1266.47
				Vertical	1054.57
			16	Orizontal	1072.42
				Vertical	1103.42

1. Cazuri de Testare

Toate tipurile de distributie (pe verticala, pe orizontala):

- 4 thread-uri pe o matrice de 10x10 si filtru de 3X3
- 2/4/8/16 thread-uri pe o matrice 1000X1000 si filtru de 5X5
- 2/4/8/16 thread-uri pe o matrice 10X10000 si filtru de 5X5

- 2/4/8/16 thread-uri pe o matrice 10000X10 si filtru de 5X5
- 2/4/8/16 thread-uri pe o matrice 10000X10000 si filtru de 5X5

-

2. Modalitate de rulare

- Pentru versiunea **Java** astfel:

```
cd D:\Facultate\Anul-3\Semestrul-I\PPD\Tema-1\Java\build\classes\java\main
.\scripJava.ps1 -JavaClass com.example.Main -Runs 10 -N 1000 -M 1000 -K 3 -
```

Pentru versiunea C++: cd D:\Facultate\Anul-3\Semestrul-I\PPD\Tema-1\C++\cmake-build-debug .\scriptC.ps1 -CppProgram ".\StaticMain.exe" -Runs 5 -N 100 -M 100 -K 3

3. Concluzii

Secvențial vs Paralel (C++):

- Implementările paralele sunt în general mai rapide decât varianta secvențială.
 - Convoluția paralelă pe **orizontală** tinde să fie mai rapidă decât cea pe verticală.
 - Pentru volume mici, secvențialul poate fi mai rapid din cauza overhead-ului thread-urilor.
- Avantajul paralelismului devine evident odată cu creșterea volumului de date.

C++ vs Java:

- C++ este semnificativ mai rapid decât Java pentru aceleași dimensiuni ale matricelor, datorită optimizărilor la nivel de compilator și controlului asupra memoriei.

Alocare dinamică vs statică (C++):

- Alocarea dinamică (`int**`) este mai lentă decât cea statică, din cauza accesului în heap și a fragmentării memoriei.
- Totuși, este necesară pentru matrici foarte mari (ex. $N = M = 10000$) pentru a evita stack overflow-ul.

General

- Paralelismul oferă cel mai mare câștig pentru volume mari de date.
- Accesul la memorie influențează performanța paralelă (orizontal vs vertical).
- C++ rămâne mai rapid decât Java, dar alocarea memoriei trebuie gestionată atent pentru matrici mari.